

MIT APP INVENTOR

HISTORY

MIT APP INVENTOR MIT App Inventor is an intuitive, visual programming environment that allows everyone even children to build fully functional apps for smartphones and tablets. Those new to MIT App Inventor can have a simple first app up and running in less than 30 minutes. And what's more, our blocks-based tool facilitates the creation of complex, high-impact apps in significantly less time than traditional programming environments.

The MIT App Inventor project seeks to democratize software development by empowering all people, especially young people, to move from technology consumption to technology creation. A small team of CSAIL staff and students, led by Professor Hal Abelson, forms the nucleus of an international movement of inventors. In addition to leading educational outreach around MIT App Inventor and conducting research on its impacts, this core team maintains the free online app development environment that serves more than 6 million registered users.

KEY CHARACTERISTICS

- **Blocks-Based Coding:** Replaces traditional text-based code with visual blocks that snap together, similar to building with LEGO, making it easy for beginners.
- **Cloud-Based & Free:** The platform is accessed through a web browser, requiring no heavy software installation.
- **Live Testing:** Users can instantly test apps on a connected smartphone or tablet using the MIT AI2 Companion app.
- **Target Audience:** Originally developed by Google and maintained by MIT, it is primarily designed for students, educators, and novice developers to create, share, and remix apps.

MIT App Inventor Sample Game Tutorial:Tic-Tac-Toe

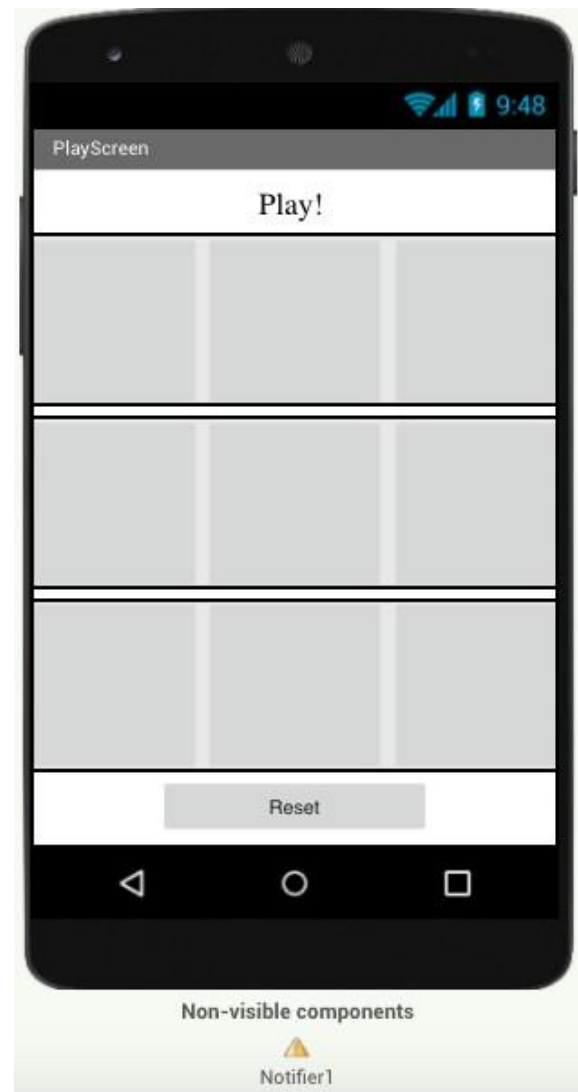
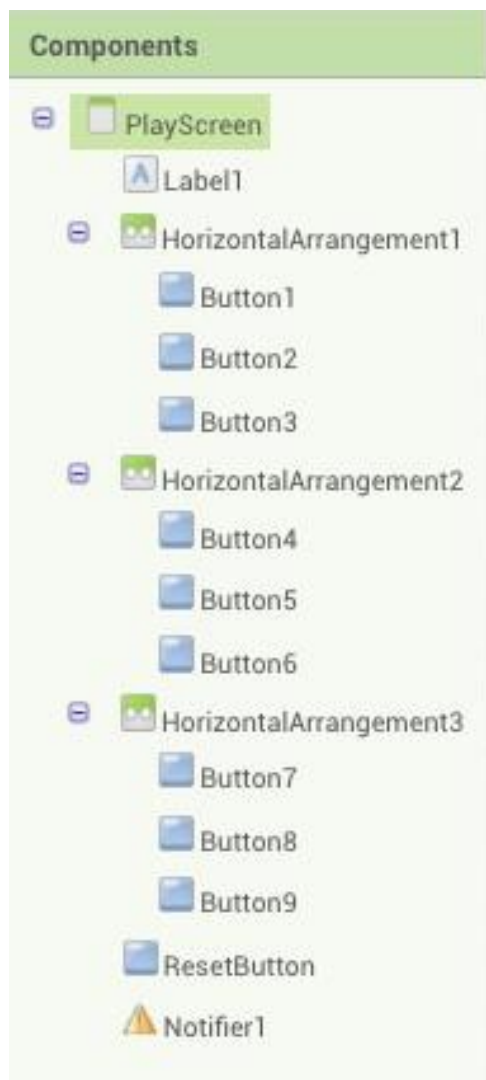
Overview

Tic-tac-toe is a well-known game that is traditionally played on a piece of paper, wherein two players fill in squares in a three-by-three grid of spaces with either a letter “X” or a letter “O”. The goal of the game is to fill in three spaces in a row, either diagonally or across the grid. Since the game itself is so simple, developing an app that simulates it should not be a terribly difficult task. In this tutorial, we will program our own tic-tac-toe app, and possibly revive any childhood memories you may have of playing this game with a sibling or friend.

This guide assumes that you have some background with MIT App Inventor and that you have it set up properly on your device.

Step 1: Design of the Play Screen

The most important part of tic-tac-toe is the three-by-three grid of spaces that players mark as their own in order to win the game. However, we will also need nine buttons for the playable spaces, a button to reset the game, a label that indicates whose turn it is, and a notifier to let us know when a game is over.



Things to note about the play screen include, but are not limited to:

- The Screen itself is center-aligned, both horizontally and vertically.
- The “Play!” Label has a font size of 22.
- The height of each Horizontal Arrangement is set to 25 percent.
- Each Button on the grid has a font size of 50, their heights are set to “Fill Parent”, and their texts are center-aligned.
- The Reset Button has a font size of 14 and is set to a width of 50 percent.
- The Notifier will be used for a pop-up that congratulates the winner of the game; we will program this soon.

Now that we have a field to play on, so to speak, we are ready to program the back-end of the game.

Step 2: Initialization of Globals

Next, we will initialize the variables that will keep track of the state of each game. There are a total of 12. They include:

- A “winner” variable, which is set to “*” at the beginning of each game
- A “turnsPassed” variable, which starts at 0
- A “turn” variable, which starts with “X” to keep it simple
- Nine total buttons that are all initialized to 0

In the blocks editor, the global initialization blocks will look something like this:

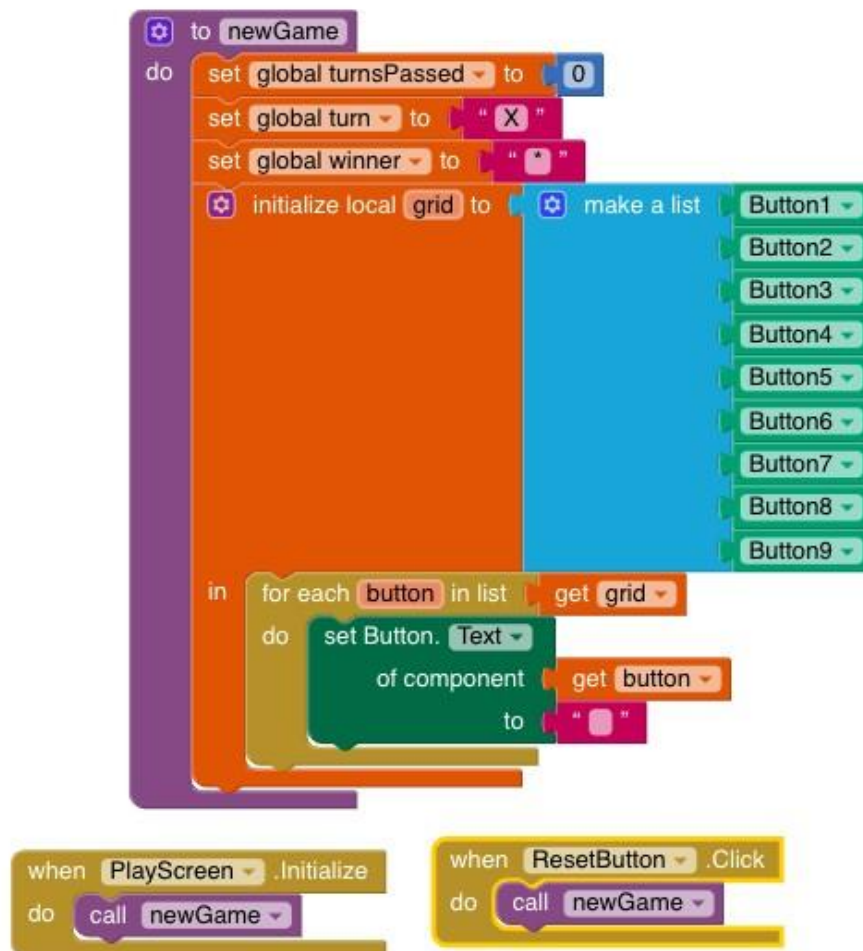


It is strongly recommended that you keep all of your global variable declarations in one place, for the sake of your own time and sanity.

Step 3: The New Game Routine

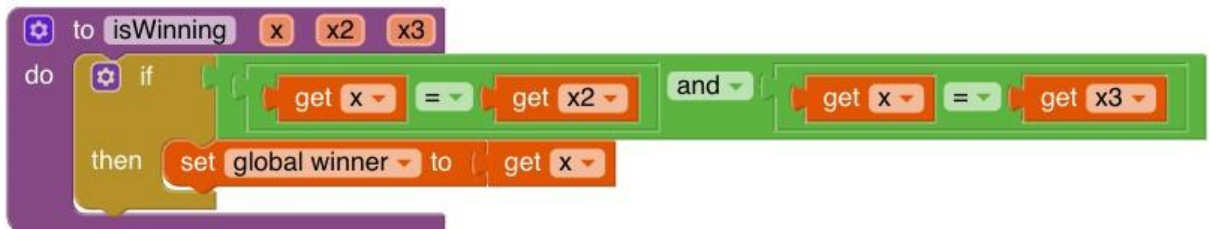
We need a function to start new games. Otherwise, we will have to re-write the same “new game” code for every possible win condition. It is much easier to just write a newGame procedure and call that procedure every time someone wins the game. We will also call this function when the screen is first initialized, and when someone hits the reset button.

The newGame procedure resets the turns passed counter, sets the current turn to “X”, resets the winner variable, puts the nine playable buttons in a list, and sets all their text to an empty string. An image is included below.



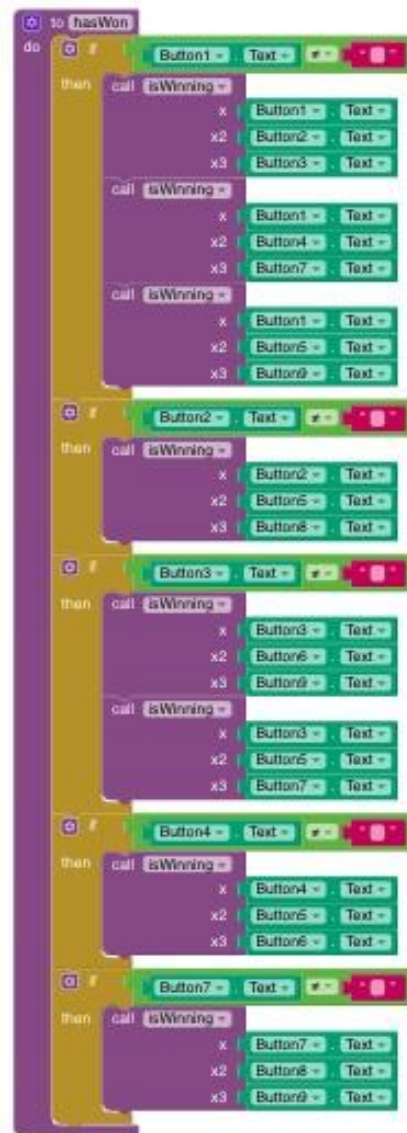
Step 4: Equality Check for 3 Button Texts

The next procedure that we should write is called `isWinning` in the image below. This procedure just checks three text variables for equality, and sets the winner to the text found in the case that they are all the same. In the next step, we will call this procedure for every possible combination of buttons that could win the game at the end of a turn. Giving the equality check its own function vastly simplifies the next one.



Step 5: Checking for a Winner

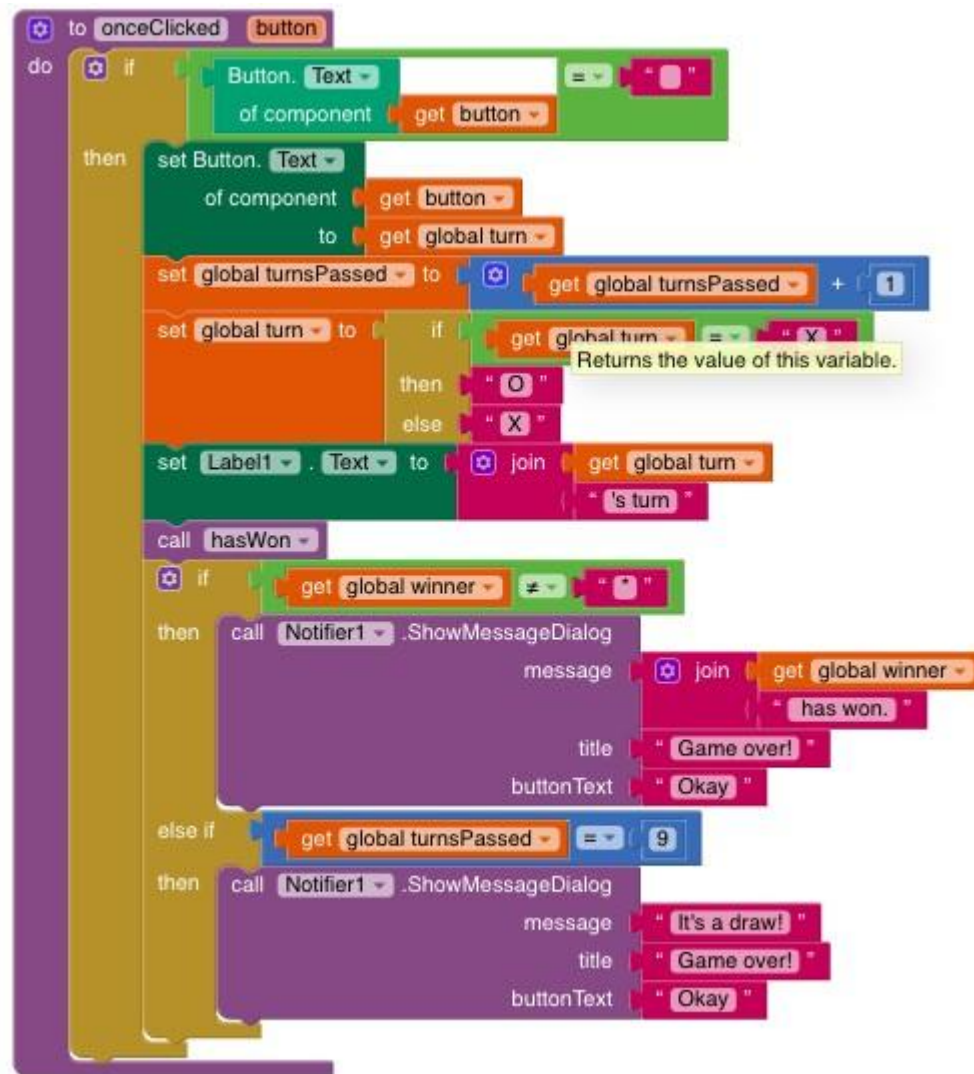
To check for a winner, we will need to call the equality check `hasWon` procedure on 8 total combinations of buttons. You can win tic-tac-toe vertically, horizontally, or diagonally, and we will need to check every possible win condition each time this procedure is called. Otherwise, we run the risk of denying victory to the winner of each game.



This is, by far, the longest procedure we have to write because of the many ways you can win the game.

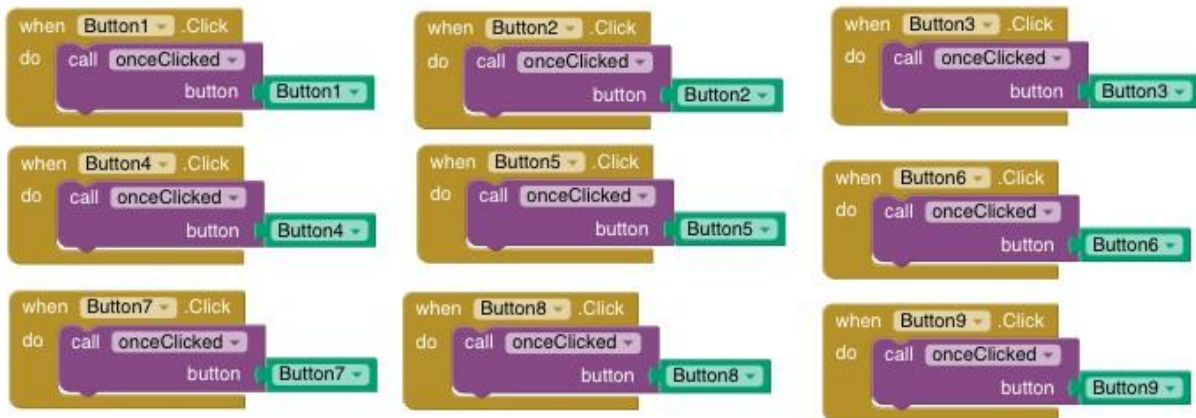
Step 6: Programming the Playable Buttons

We are missing a procedure that handles the pressing of the playable buttons. Each time a button is pressed, we want to ensure that it hasn't already been played, call the win-check procedure, announce a winner if the game has been won, and reset the game after the victory ceremonies are complete.



The procedure itself needs to be called any time someone presses one of the nine playable buttons. Sadly, there is no way to program a handler for all

nine buttons in a single block of code, so we have to just write nine blocks of code- one for each button.



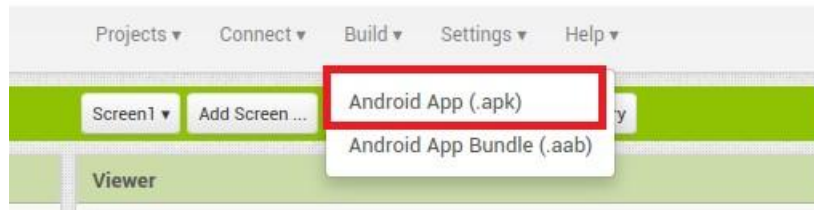
Once the grid of button handlers is programmed, congratulations! You are ready to test your tic-tac-toe app!

Known Issues/Suggestions

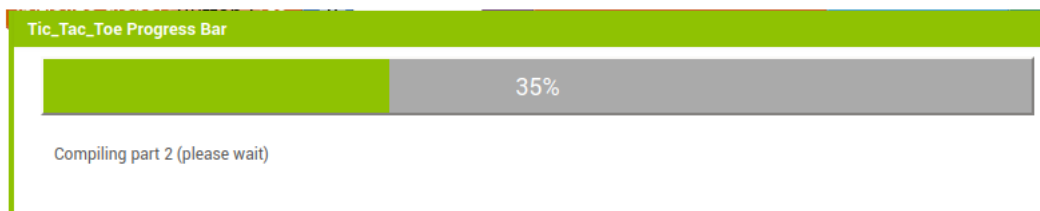
- The game does not announce whose turn it is at the start of the game, even though it should always be X's turn at the start.
- We can probably afford to set X's text color to red and O's text color to blue, but whether that is necessary is a subject matter.

Step 7: Testing

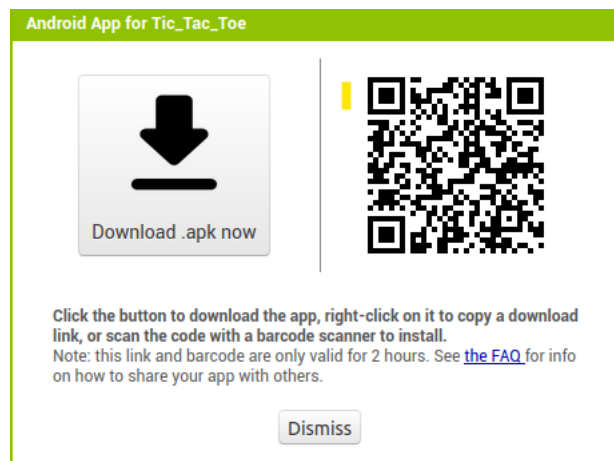
Now in testing part we are going to test the application by following the below steps.



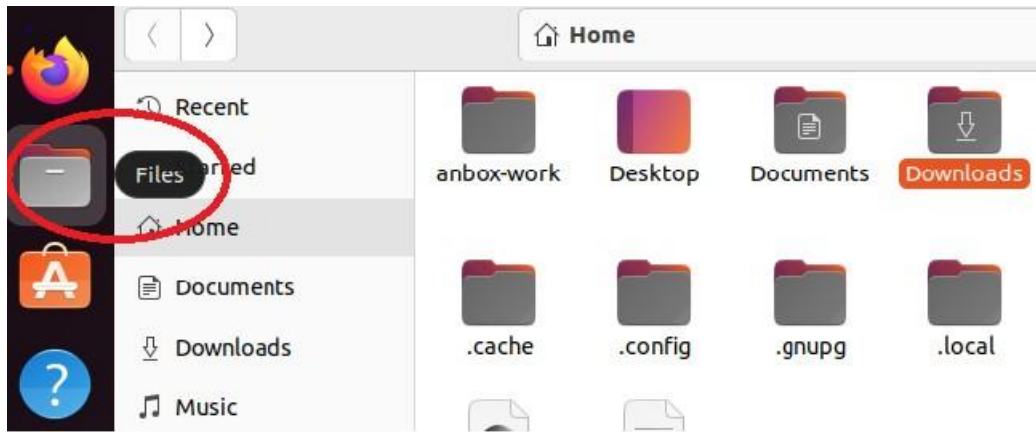
You need to click on the option “Build” at the top of MIT App Inventor. Then, you get the above options and click on the Android App (.apk) to download the apk file.



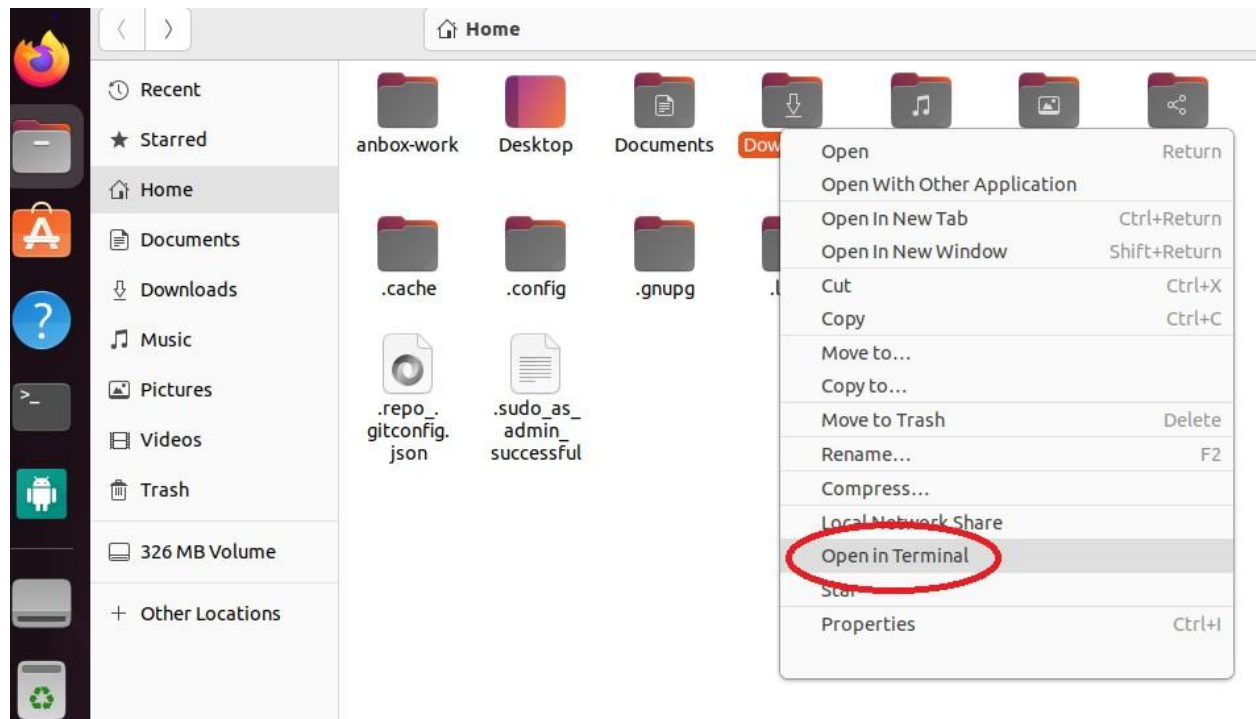
Once you click on the Android App (.apk), you will get the above popup, which shows the compiling percentage



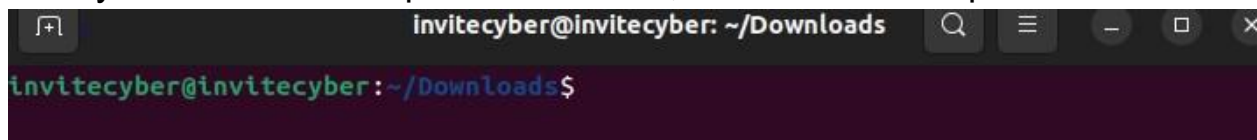
Click on the “Download.apk now” to download the apk file.



Click on the “Files”, then you will get different folders and given right click on the downloads folder.



Once you click on the “Open in Terminal”, terminal will be opened as below

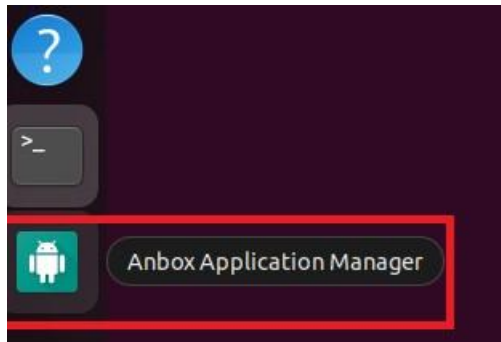


Give the below command in the terminal to run the apk file

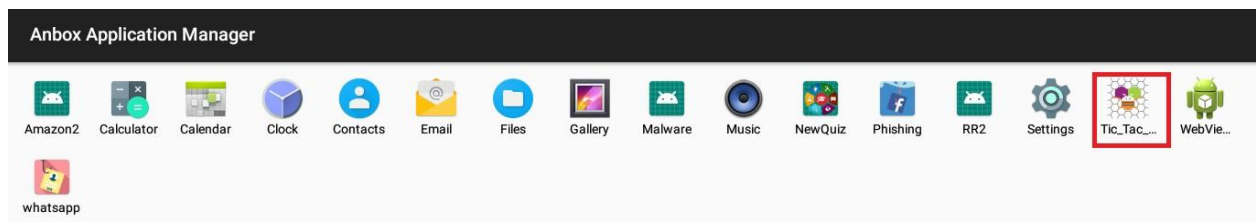
\$ adb install filename.apk

```
invitecyber@invitecyber: ~/Downloads
invitecyber@invitecyber:~/Downloads$ adb install Tic_Tac_Toe.apk
Performing Streamed Install
Success
invitecyber@invitecyber:~/Downloads$
```

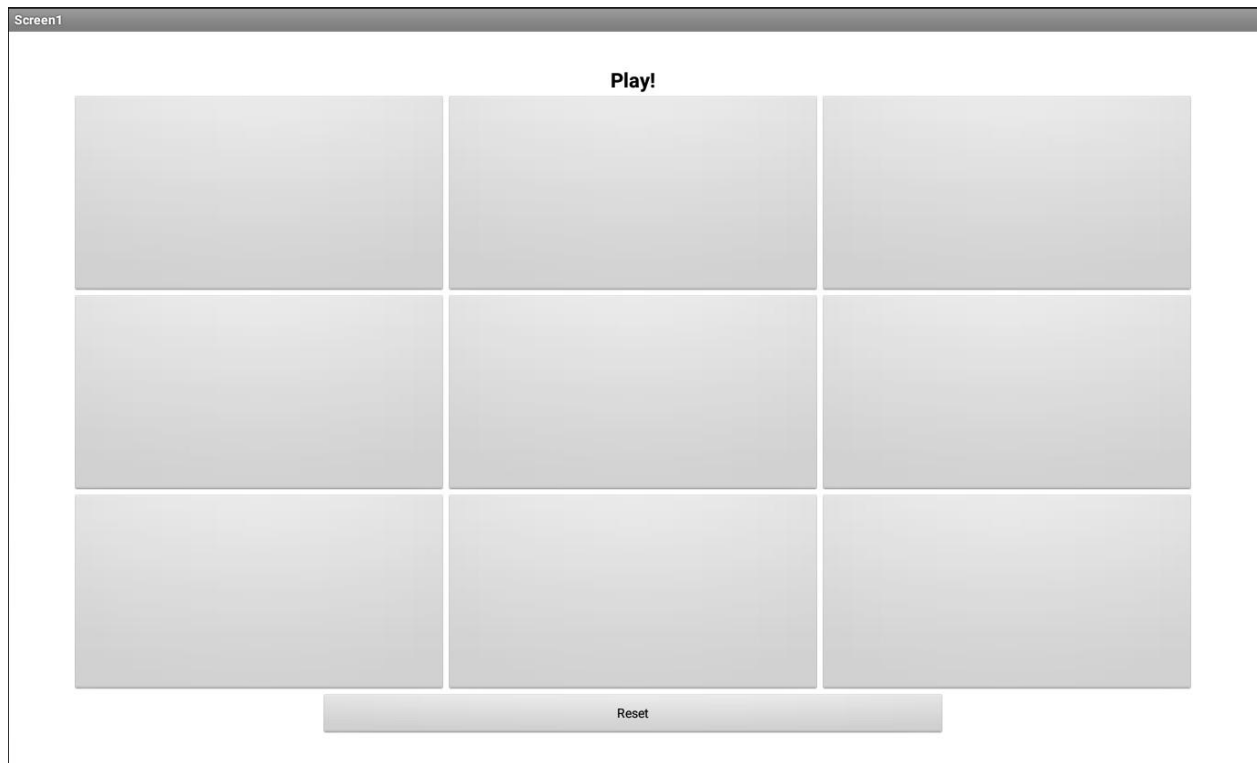
Give the command and click on ENTER



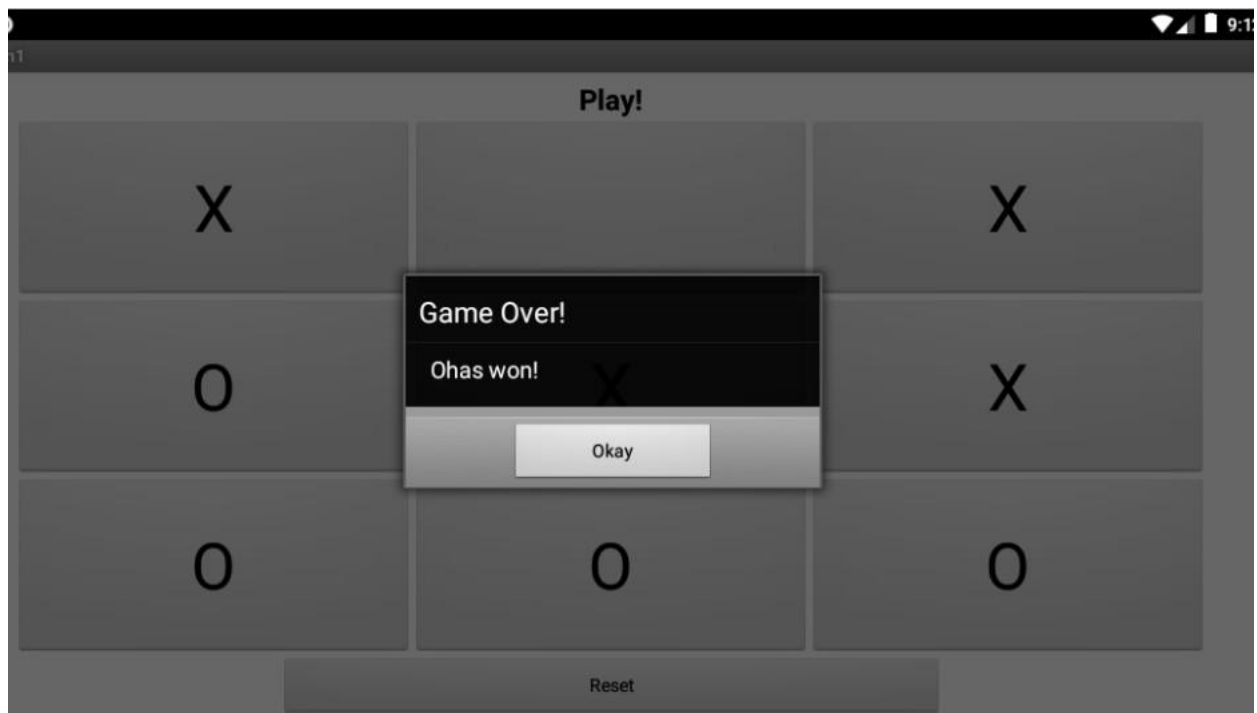
Once the app is installed click on the “Anbox Application Manager” to start the application installed.



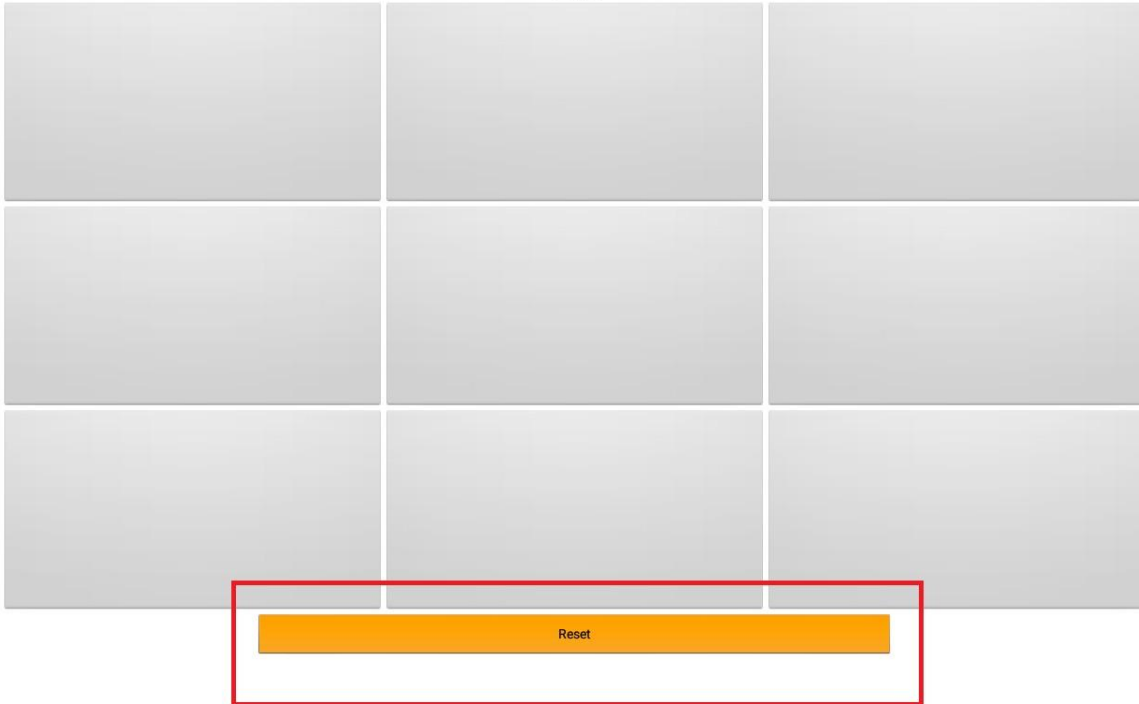
App installed is shown as above, you just need to DoubleClick on the application, so that Screen is opened as below



Click on the buttons and start playing it!!!!



Screen1

Play!

Click on the reset button, if you want to play a new game!